Aegis — A Measurement-First Execution Harness

Running an AI security benchmark reproducibly — and building a harness that catches its own lies.

The problem

The hard part of evaluating AI security agents isn't the model — it's producing measurements you can trust. Public benchmarks grade pass/fail on ad-hoc harnesses, and a single silent bug — a stale log, an empty input, a non-comparable environment — quietly turns a result into noise without anyone noticing. Aegis's execution harness, which runs the BountyBench real-CVE benchmark, applies the same discipline as its verifier: **don't trust, verify — at the infrastructure layer.**

Measure before optimizing

Before touching performance, I profiled 93 runs. It overturned three confident assumptions at once: the LLM calls were only **29%** of wall time (not the bottleneck), the Docker tasks were the *fast* run type, and per-run environment *setup* was the real cost. That single profile killed a multi-node build I was about to start — it would have spent days speeding up a part that was already fast. The lesson the whole harness is built on: almost every win came from measuring instead of guessing.

A harness that catches its own lies

The load-bearing features exist specifically to stop the harness from deceiving itself — the part most benchmark infrastructure skips:

- **Preflight + environment fingerprint:** a run whose environment (image, patches, container state) doesn't match the recorded baseline is invalidated — so results stay comparable instead of silently drifting.
- **Freshness-gated logs:** caught a class of bug where a 6-second run was recorded as a clean 18,000-token result, because the collector picked up a stale log.
- **Oracle-injection logging:** caught an empty input that had silently produced a false "the method doesn't work" result — one report away from publishing a wrong conclusion.
- **A comparability check that *refused* a shortcut:** the agent already receives its target host, so adding a stronger hint would have beaten the published baseline unfairly. I left it out and reported the honest number.

The debugging journey (where the real work was)

Symptom	Real root cause	Fix
Everything 5-10× slow / flaky	ARM64 agent image run under qemu emulation on an amd64 host	Native amd64 rebuild
~58% of all task failures	Root-owned files from containers breaking `git clean` (not a "lock race")	One pre-cleanup chown
Run crashes, disk full	Container runtime (containerd) writing layers to the wrong disk	Symlink to the data disk

These are the subtle, infrastructure-real bugs that decide whether a benchmark run is *data* or *noise* — and none of them are visible from the model side.

What it produced

A reproducible, single-node harness for a 40-task real-CVE security benchmark, with a bare-agent baseline in the published range (Exploit ~27%, Detect ~3%) and a clean, definitive *negative* result on the localization hypothesis the system was built to test — the kind of honest finding measurement rigor is supposed to surface.

Honest limits

Stated, not hidden. Execution is **single-node**: the distributed multi-node version was designed and its per-instance isolation validated, but never deployed — a cloud free-tier quota cap hard-limits the project to one machine. The baseline is preliminary: the first full run had a high infrastructure-failure rate (since traced to the two root causes above and fixed). The architecture is a deliberate persistent-VM approximation of the gold standard (ephemeral container-per-run CI), with the trade-offs documented.

Stack & method. Python, Docker / Kali, Google Cloud (Compute Engine), BountyBench (real-CVE benchmark). Process-level isolation, deterministic preflight + environment fingerprinting, freshness-gated logging, and profiling-driven optimization. Companion to the verifier pillar; the throughline across both — build the measurement layer first, trust the numbers it produces, and make the system catch its own lies.